Max Profit Finder: Divide and Conquer vs Linear Scan

CS314 · Analysis of Algorithms · Mini Project (Divide and Conquer)

1. Problem

Given the closing prices `p[0..n-1]` of a stock on n consecutive days, choose a buy day i and a sell day j > i that maximise the profit p[j] − p[i]. If no pair gives a positive profit, the answer is “no trade” with profit 0. The input is a price CSV (column `price`); the output is (buy day, sell day, profit). Checking every pair costs Θ(n²), so the goal is to do better by splitting the problem in half, and to compare the result against a linear scan.

2. Graph Model

Model the days as the vertices 0, 1, …, n−1 of a path graph. The edge (k, k+1) carries the weight d[k] = p[k+1] − p[k], the price change of day k+1. A trade that buys on day i and sells on day j is the subpath from vertex i to vertex j, and its weight telescopes:

$$d[i] + d[i+1] + \dots + d[j-1] = p[j] - p[i]$$

So the best trade is the maximum-weight subpath of the path graph, that is, the maximum subarray of d. For p = [5, 1, 3, 4, 9, 2] we get d = [−4, 2, 1, 5, −7]; the best subarray is [2, 1, 5] with sum 8, so buy on day 1 and sell on day 4. The figure shows the price graph of the 1,000-day sample file with the trade found.

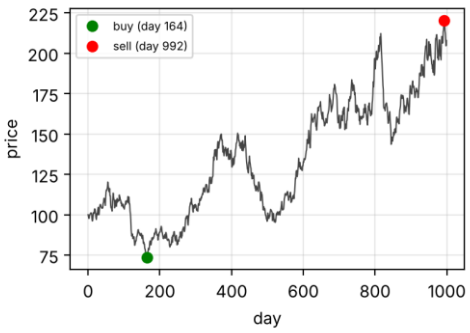


Figure 1. Price graph and the best trade

3. Algorithm and Complexity

Divide. Compute d, then split d[lo..hi] at mid = ⌊ (lo + hi)/2⌋ . **Conquer.** Find the best subarray of d[lo..mid] and of d[mid+1..hi] recursively; a single element is the base case. **Combine.** Find the best subarray that crosses the middle: the best suffix of the left half plus the best prefix of the right half, found in one pass outwards from mid. The answer is the largest of the three candidates. A subarray d[s..e] gives buy day s and sell day e + 1.

```
MAX-SUBARRAY(d, lo, hi)
  if lo == hi: return (lo, hi, d[lo])
  mid = (lo + hi) // 2
  left, right = MAX-SUBARRAY(d, lo, mid), MAX-SUBARRAY(d, mid+1, hi)
  cross = MAX-CROSSING(d, lo, mid, hi)
  return the one of left, right, cross with the largest sum
```

Correctness. Every subarray of d[lo..hi] lies completely in the left half, completely in the right half, or contains both mid and mid+1. The three cases are covered by the two recursive calls and by MAX-CROSSING, so the largest of the three is the global maximum.

Running time. Dividing costs Θ(1), MAX-CROSSING costs Θ(n), and there are two half-size sub-problems, so T(n) = 2T(n/2) + Θ(n). Here a = 2, b = 2 and f(n) = Θ(n) = Θ(n^($\log_2$ 2)), which is case 2 of the Master Theorem, so T(n) = Θ(n log n). The recursion tree agrees: $\log_2$ n levels, each doing Θ(n) work in total. Building d costs Θ(n), and the space is Θ(n) for d plus Θ(log n) for the recursion stack. The linear scan keeps the lowest price seen so far and tests the profit of selling today, which gives Θ(n) time and Θ(1) space. Since every price must be read at least once, Θ(n) is optimal.

Algorithm	Time	Space	Recurrence / idea
Brute force	Θ(n²)	Θ(1)	try all n(n−1)/2 pairs (i, j)
Divide and conquer	Θ(n log n)	Θ(n)	T(n) = 2T(n/2) + Θ(n)
Linear scan	Θ(n)	Θ(1)	track the lowest price seen so far

4. Experiments

Setup: Python 3.12, time.perf_counter, best of 5 runs, prices generated as a random walk with N(0, 1) steps, n from 10³ to 10⁶. Brute force was run only for n ≤ 5,000. For every input the three algorithms returned the same profit; the sample file gives buy day 164 (73.39), sell day 992 (220.30), profit 146.91 for both algorithms. Correctness is also covered by 14 unit tests, including 500 random inputs compared against brute force.

n	D&C (ms)	Linear (ms)	Brute (ms)	D&C / linear
1,000	1.42	0.064	23.2	22.0
5,000	11.74	0.499	534.0	23.5
10,000	15.81	0.602	–	26.3
100,000	168.79	5.938	–	28.4
500,000	863.11	28.792	–	30.0
1,000,000	1631.98	50.689	–	32.2

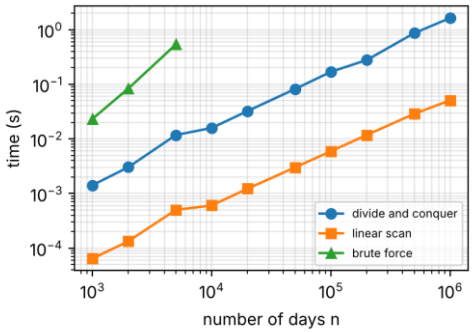


Figure 2. Running time (log–log)

Observations. The time of the linear scan divided by n stays between about 50 and 65 ns, and the time of divide and conquer divided by n $\log_2$ n falls from about 140 ns at n = 10^3 to about 80 ns at n = 10^6, so both curves follow their predicted growth. Brute force grows as n^2: at n = 5,000 it needs 534 ms, against 11.7 ms and 0.5 ms for the other two. Divide and conquer was 22 to 32 times slower than the linear scan at every size. Part of this gap is the extra log n factor, and part is the constant overhead of Python function calls and of building the difference list.

5. Conclusion

The maximum profit is the maximum subarray of the daily price changes, and divide and conquer solves it in Θ(n log n) by combining the best trades in the left half, in the right half and across the middle. Both it and the linear scan are exact and agree with brute force on all tests. The linear scan is the better tool for this problem: it is Θ(n), uses constant memory, and was 22 to 32 times faster in the experiments. The value of the divide-and-conquer version is that it follows the paradigm of the lecture (divide, conquer, combine, Master Theorem), and that its two halves are independent sub-problems.